

在 Google Kubernetes Engine 部署及更新 .NET Core 應用程式

來源：<https://codelabs.developers.google.com/codelabs/dotnet-kubernetes?hl=zh-tw>

步驟數：9

目錄

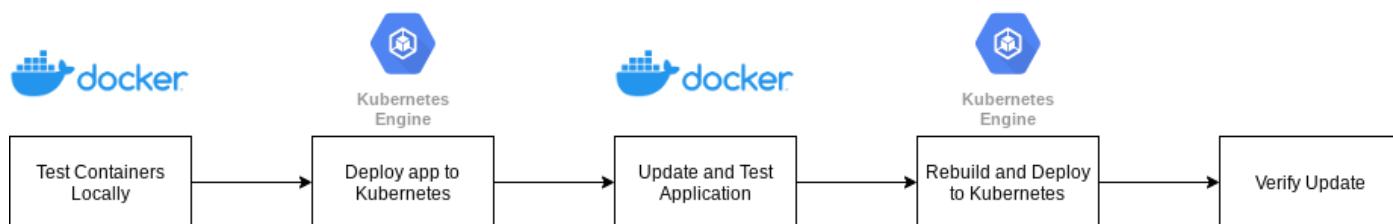
1. [總覽](#)
2. [設定和需求](#)
3. [在本機測試並確認所需功能](#)
4. [使用「網頁預覽」功能從 Cloud Shell 存取服務](#)
5. [部署到 Kubernetes](#)
6. [修改應用程式](#)
7. [重新部署更新後的應用程式](#)
8. [恭喜！](#)
9. [清除所用資源](#)

1. 總覽

Microsoft [.NET Core](#) 是 .NET 的開放原始碼跨平台版本，可在容器中以原生方式執行。[.NET Core](#) 可在 GitHub 上取得，並由 Microsoft 和 .NET 社群維護。本實驗室會將容器化的 [.NET Core](#) 應用程式部署至 [Google Kubernetes Engine](#) (GKE)。

本實驗室遵循典型的開發模式，也就是在開發人員的本機環境中開發應用程式，然後部署至正式環境。在本實驗室的第一部分，您會使用 [Cloud Shell](#) 中執行的容器，驗證 [.NET Core](#) 範例應用程式。驗證完成後，應用程式就會使用 GKE 部署至 [Kubernetes](#)。本實驗室包含建立 GKE 叢集的步驟。

在本實驗室的第二部分，我們會對應用程式進行小幅變更，顯示執行該應用程式例項的容器主機名稱。接著，在 [Cloud Shell](#) 中驗證更新後的應用程式，並更新部署項目以使用新版本。下圖說明本實驗室的活動順序：



費用

如果您完全按照本實驗室的內容操作，系統會收取下列服務的一般費用

- [Google Kubernetes Engine 定價](#)
 - [Google Container Registry 價格](#)
-

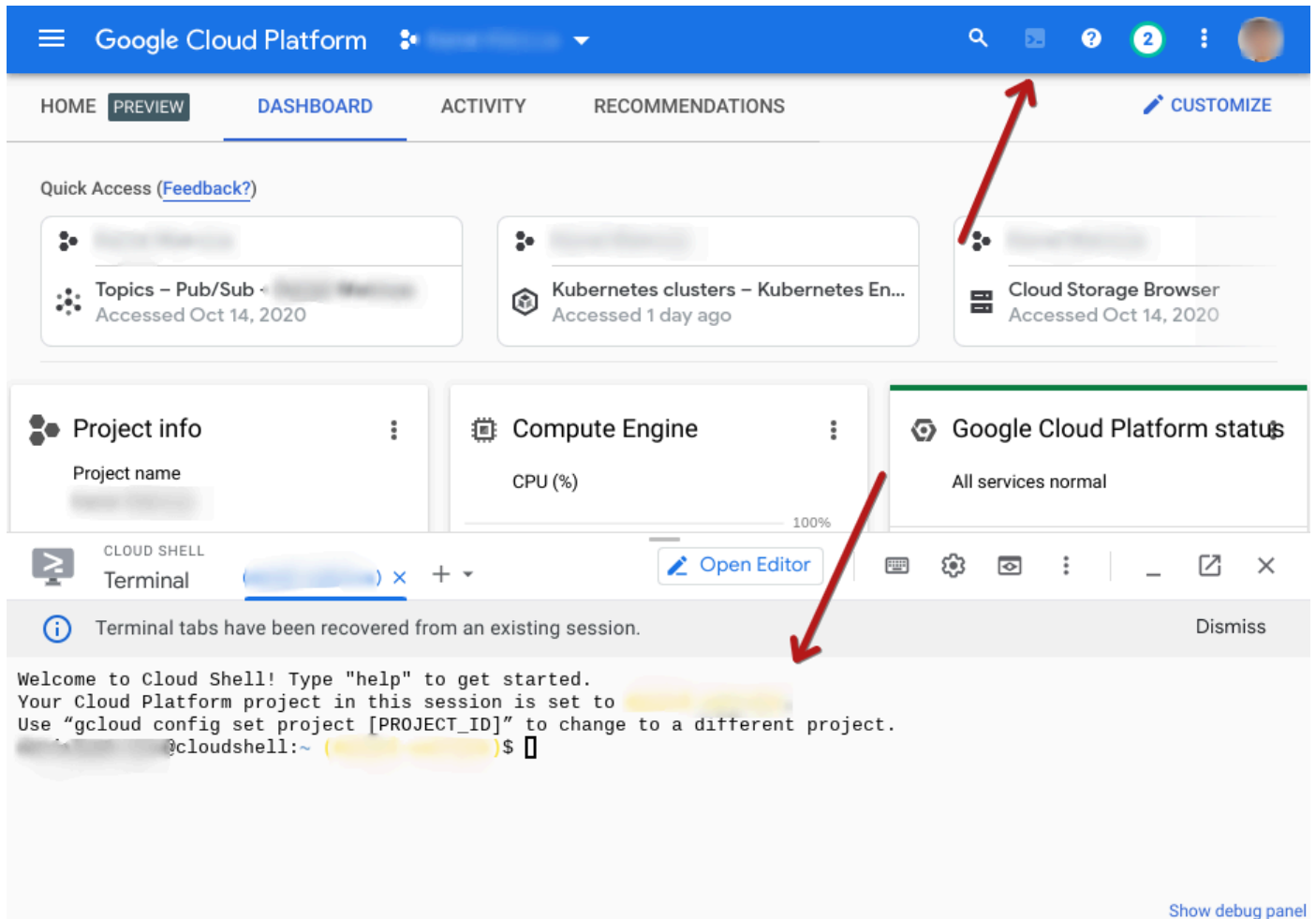
步驟 2 · 約 5 分鐘

2. 設定和需求

必要條件

如要完成本實驗室，請務必備妥 Google Cloud 帳戶和專案。如需建立新專案的詳細操作說明，請參閱[這個程式碼研究室](#)。

本實驗室會使用 Cloud Shell 中執行的 Docker，您可以透過 Google Cloud 控制台存取 Cloud Shell，其中已預先設定許多實用工具，例如 gcloud 和 Docker。存取 Cloud Shell 的方式如下。按一下右上角的 Cloud Shell 圖示，即可在控制台視窗的底部窗格中顯示 Cloud Shell。



GKE 叢集的替代設定選項 (選用)

本實驗室需要 Kubernetes 叢集。在下一節中，我們將建立具有簡單設定的 GKE 叢集。本節列出一些 gcloud 指令，提供使用 GKE 建構 Kubernetes 叢集時可用的替代設定選項。舉例來說，您可以使用下列指令識別不同的機器類型、可用區，甚至是 GPU (加速器)。

- 使用 `gcloud compute machine-types list` 指令列出機器類型
- 使用下列指令列出 GPU `gcloud compute accelerator-types list`
- 使用這個指令列出運算可用區：`gcloud compute zones list`
- 取得任何 gcloud 指令的說明 `gcloud container clusters --help`
 - 舉例來說，這會提供建立 Kubernetes 叢集的詳細資料 `gcloud container clusters create --help`

如需 GKE 的完整設定選項清單，請參閱[這份文件](#)。

準備建立 Kubernetes 叢集

在 Cloud Shell 中，您必須設定部分環境變數，並設定 gcloud 用戶端。方法是使用下列指令。

請務必使用自己的 `YOUR_PROJECT_ID` 修改匯出指令。您也可以視需要變更區域。

```
export PROJECT_ID=YOUR_PROJECT_ID
export DEFAULT_ZONE=us-central1-c

gcloud config set project ${PROJECT_ID}
gcloud config set compute/zone ${DEFAULT_ZONE}
```

請注意，如果 Cloud Shell 工作階段逾時，您必須重新連線，並再次執行上述指令。這是因為本實驗室中的許多其他指令，都會假設已設定這些環境變數和 gcloud config 設定。

建立 GKE 叢集

由於本實驗室會在 Kubernetes 上部署 .NET Core 應用程式，因此必須建立叢集。使用下列指令，在 Google Cloud 中透過 GKE 建立新的 Kubernetes 叢集。

```
gcloud container clusters create dotnet-cluster \
  --zone ${DEFAULT_ZONE} \
  --num-nodes=1 \
  --node-locations=${DEFAULT_ZONE},us-central1-b \
  --enable-stackdriver-kubernetes \
  --machine-type=n1-standard-1 \
  --workload-pool=${PROJECT_ID}.svc.id.goog \
  --enable-ip-alias
```

- `--num-nodes` 是每個區域要新增的節點數量，之後可以調整
- `--node-locations` 是以半形逗號分隔的區域清單。在本例中，您在上述環境變數中識別出的區域和 `us-central1-b` 會用於
 - 注意：這份清單不得含有重複項目
- `--workload-pool` 建立工作負載身分，讓 GKE 工作負載可以存取 Google Cloud 服務

叢集建構期間，系統會顯示以下內容

```
Creating cluster dotnet-cluster in us-central1-b... Cluster is being deployed...:
```

設定 kubectl

`kubectl` CLI 是與 Kubernetes 叢集互動的主要方式。如要搭配剛建立的新叢集使用，必須設定該工具，以便對叢集進行驗證。請使用下列指令執行這項操作。

```
$ gcloud container clusters get-credentials dotnet-cluster --zone ${DEFAULT_ZONE}
Fetching cluster endpoint and auth data.
kubeconfig entry generated for dotnet-cluster.
```

現在應該可以使用 `kubectl` 與叢集互動。

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
gke-dotnet-cluster-default-pool-02c9dcb9-fgxj	Ready	<none>	2m15s	v1.16.13-gke.401
gke-dotnet-cluster-default-pool-ed09d7b7-xdx9	Ready	<none>	2m24s	v1.16.13-gke.401

步驟 3 · 約 2 分鐘

3. 在本機測試並確認所需功能

本實驗室使用 Docker Hub 上 .NET 官方存放區的下列容器映像檔。

- https://hub.docker.com/_/microsoft-dotnet
- https://hub.docker.com/_/microsoft-dotnet-samples

在本機執行容器，驗證功能

在 Cloud Shell 中執行下列 Docker 指令，確認 Docker 正常運作，且 .NET 容器可正常運作：

Hello from .NET!

Environment:

```
Linux 5.4.58-07649-ge120df5deade #1 SMP PREEMPT Wed Aug 26 04:56:33 PDT 2020
```

您也可以在此 Cloud Shell 中驗證範例網頁應用程式。下列 Docker 執行指令會建立新的容器，公開通訊埠 80，並將該通訊埠對應至 localhost 通訊埠 8080。請注意，localhost 在本例中位於 Cloud Shell。


```
$ docker run -it --rm -p 8080:80 --name aspnetcore_sample
mcr.microsoft.com/dotnet/samples:aspnetapp
warn: Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository[60]
      Storing keys in a directory '/root/.aspnet/DataProtection-Keys' that may not be
persisted outside of the container. Protected data will be unavailable when container is
destroyed.
warn: Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager[35]
      No XML encryptor configured. Key {64a3ed06-35f7-4d95-9554-8efd38f8b5d3} may be
persisted to storage in unencrypted form.
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://[::]:80
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /app
```

由於這是網頁應用程式，因此必須在網頁瀏覽器中查看及驗證。下一節將說明如何在 Cloud Shell 中使用網頁預覽功能執行這項操作。

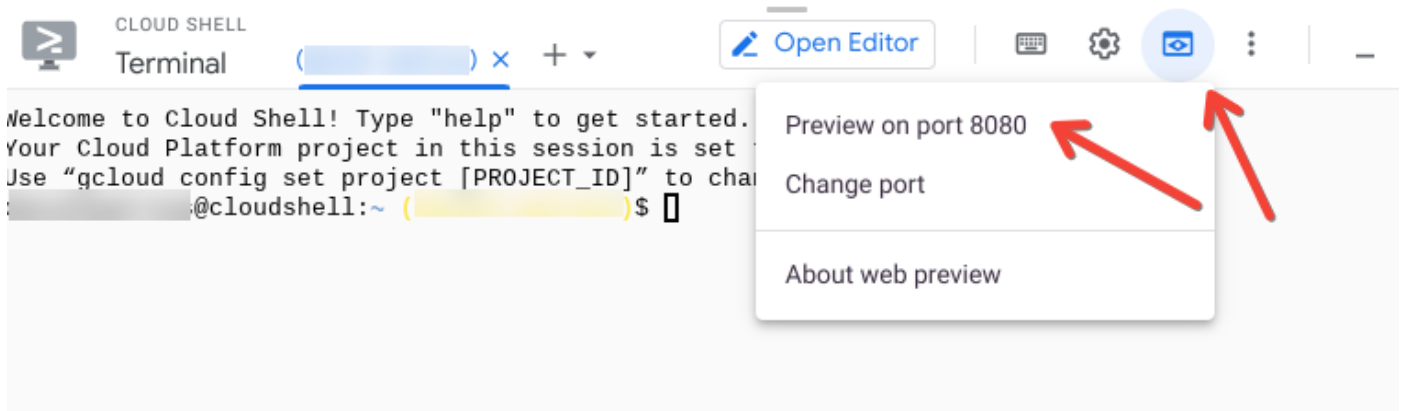
步驟 4 · 約 2 分鐘

4. 使用「網頁預覽」功能從 Cloud Shell 存取服務

Cloud Shell 提供「網頁預覽」功能，可讓您使用瀏覽器與在 Cloud Shell 執行個體中執行的程序互動。

使用「網頁預覽」功能在 Cloud Shell 中查看應用程式

在 Cloud Shell 中，按一下「網頁預覽」按鈕，然後選擇「透過以下通訊埠預覽：8080」(或網頁預覽設定使用的任何通訊埠)。



系統會開啟瀏覽器視窗，並顯示類似下列的網址：

```
https://8080-cs-754738286554-default.us-central1.cloudshell.dev/?authuser=0
```

「變更通訊埠」選項可用於將服務在 Cloud Shell 中執行的通訊埠，與「網頁預覽」使用的通訊埠相符。

使用網頁預覽功能查看 .NET 範例應用程式

啟動「網頁預覽」並載入提供的網址，即可查看上一個步驟啟動的範例應用程式。如下所示：

Welcome to .NET

Environment

.NET 5.0.1-servicing.20575.16

Linux 5.4.49+ #1 SMP Mon Nov 30 19:42:49 PST 2020

Metrics

Containerized	true
CPU cores	4
cgroup memory usage	32059392
Memory, current usage (bytes)	78860288
Memory, max available (bytes)	9223372036854771712

步驟 5 · 約 3 分鐘

5. 部署到 Kubernetes

建構 YAML 檔案並套用

下一個步驟需要一個 YAML 檔案，其中說明兩項 Kubernetes 資源：Deployment 和 Service。在 Cloud Shell 中建立名為 `dotnet-app.yaml` 的檔案，並在其中加入下列內容。

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dotnet-deployment
  labels:
    app: dotnetapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: dotnetapp
  template:
    metadata:
      labels:
        app: dotnetapp
    spec:
      containers:
        - name: dotnet
          image: mcr.microsoft.com/dotnet/samples:aspnetapp
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: dotnet-service
spec:
  selector:
    app: dotnetapp
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 80
```

現在請使用 `kubectl` 將這個檔案套用至 Kubernetes。

```
$ kubectl apply -f dotnet-app.yaml
deployment.apps/dotnet-deployment created
service/dotnet-service created
```

請注意訊息，確認所需資源已建立。

探索產生的資源

我們可以透過 `kubectl` CLI 檢查上述建立的資源。首先，請查看 Deployment 資源，確認新的 Deployment 是否存在。

```
$ kubectl get deployment
NAME                READY    UP-TO-DATE    AVAILABLE    AGE
dotnet-deployment   3/3      3              3             80s
```

接著查看 ReplicaSet。上述部署作業應會建立 ReplicaSet。

```
$ kubectl get replicaset
NAME                                DESIRED    CURRENT    READY    AGE
dotnet-deployment-5c9d4cc4b9       3          3          3        111s
```

最後，請查看 Pod。Deployment 指出應該有三個執行個體。下列指令應會顯示有三個執行個體。新增 `-o wide` 選項，以便顯示執行這些執行個體的節點。

```
$ kubectl get pod -o wide
NAME                                READY    STATUS    RESTARTS    AGE    IP            NODE
                                NOMINATED NODE    READINESS GATES
dotnet-deployment-5c9d4cc4b9-cspqd 1/1      Running   0            2m25s  10.16.0.8     gke-
dotnet-cluster-default-pool-ed09d7b7-xdx9 <none>    <none>
dotnet-deployment-5c9d4cc4b9-httw6 1/1      Running   0            2m25s  10.16.1.7     gke-
dotnet-cluster-default-pool-02c9dcb9-fgxj <none>    <none>
dotnet-deployment-5c9d4cc4b9-vvdln 1/1      Running   0            2m25s  10.16.0.7     gke-
dotnet-cluster-default-pool-ed09d7b7-xdx9 <none>    <none>
```

查看 Service 資源

Kubernetes 中的 [Service](#) 資源是負載平衡器。端點是由 Pod 上的標籤決定。這樣一來，只要透過上述 `kubectl scale deployment` 作業將新 Pod 新增至 Deployment，產生的 Pod 就能立即處理該 Service 的要求。

下列指令應會顯示服務資源。

```
$ kubectl get svc
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
dotnet-service      ClusterIP     10.20.9.124   <none>         8080/TCP   2m50s
...
```

您可以使用下列指令，查看 Service 的更多詳細資料。

```
$ kubectl describe svc dotnet-service
Name:                dotnet-service
Namespace:           default
Labels:              <none>
Annotations:         <none>
Selector:            app=dotnetapp
Type:                ClusterIP
IP:                  10.20.9.124
Port:                <unset> 8080/TCP
TargetPort:          80/TCP
Endpoints:           10.16.0.7:80,10.16.0.8:80,10.16.1.7:80
Session Affinity:    None
Events:              <none>
```

請注意，Service 的類型為 `ClusterIP`。也就是說，叢集內的任何 Pod 都能將 Service 名稱 `dotnet-service` 解析為 IP 位址。傳送至服務的要求會負載平衡至所有執行個體 (Pod)。上方的 `Endpoints` 值會顯示目前可供這項服務使用的 Pod IP。與上述輸出內容中的 Pod IP 進行比較。

確認正在執行的應用程式

此時應用程式已上線，可以處理使用者要求。如要存取，請使用 Proxy。下列指令會建立本機 Proxy，接受通訊埠 `8080` 的要求，並將要求傳送至 Kubernetes 叢集。

```
$ kubectl proxy --port 8080
Starting to serve on 127.0.0.1:8080
```

現在請使用 Cloud Shell 中的「網頁預覽」功能存取網頁應用程式。

如需網頁預覽操作說明，請參閱步驟 4「使用『網頁預覽』從 Cloud Shell 存取服務」

在網頁預覽產生的網址中加入 `/api/v1/namespaces/default/services/dotnet-service:8080/proxy/`。最終看起來會像這樣：

```
https://8080-cs-473655782854-default.us-central1.cloudshell.dev/api/v1/namespaces/default/services/dotnet-service:8080/proxy/
```

恭喜！您已在 Google Kubernetes Engine 上部署 .NET Core 應用程式。接著，我們將變更應用程式並重新部署。

步驟 6 · 約 5 分鐘

6. 修改應用程式

在本節中，應用程式會經過修改，以顯示執行例項的主機。這樣就能確認負載平衡是否正常運作，以及可用的 Pod 是否如預期般回應。

取得原始碼

```
git clone https://github.com/dotnet/dotnet-docker.git
cd dotnet-docker/samples/aspnetapp/
```

更新應用程式，加入主機名稱

```
vi aspnetapp/Pages/Index.cshtml
```

```
<tr>
    <td>Host</td>
    <td>@Environment.MachineName</td>
</tr>
```

建構新的容器映像檔並在本機測試

使用更新後的程式碼建構新的容器映像檔。

```
docker build --pull -t aspnetapp:alpine -f Dockerfile.alpine-x64 .
```

如先前所述，請在本機測試新應用程式

```
$ docker run --rm -it -p 8080:80 aspnetapp:alpine
warn: Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository[60]
      Storing keys in a directory '/root/.aspnet/DataProtection-Keys' that may not be
persisted outside of the container. Protected data will be unavailable when container is
destroyed.
warn: Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager[35]
      No XML encryptor configured. Key {f71feb13-8eae-4552-b4f2-654435fff7f8} may be
persisted to storage in unencrypted form.
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://[::]:80
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /app
```

與先前一樣，您可以使用「網頁預覽」存取應用程式。這次應該會看到「主機」參數，如下所示：

Home page - aspnetapp x +

8080-cs-473655782854-default.us-central1.cloudshell.dev/?authuser=0

aspnetapp Home Privacy

Welcome to .NET

Environment

.NET 5.0.1-servicing.20575.16

Linux 5.4.49+ #1 SMP Mon Nov 30 19:42:49 PST 2020

Metrics

Containerized	true
CPU cores	4
Host	ab85ce11aecd
cgroup memory usage	31502336
Memory, current usage (bytes)	72486912
Memory, max available (bytes)	9223372036854771712

© 2019 - aspnetapp - [Privacy](#)

在 Cloud Shell 中開啟新分頁，然後執行 `docker ps`，確認容器 ID 與上方顯示的「主機」值相符。

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
ab85ce11aecd	aspnetapp:alpine	"/.aspnetapp"	2 minutes ago	Up 2 minutes
0.0.0.0:8080->80/tcp	relaxed_northcutt			

為映像檔加上標記並推送，讓 Kubernetes 可以使用

映像檔必須經過標記並推送，Kubernetes 才能提取。首先列出容器映像檔，然後找出所需映像檔。

```
$ docker image list
```

REPOSITORY	TAG	IMAGE ID
aspnetapp	alpine	95b4267bb6d0
days ago	110MB	6

接著，為該映像檔加上標記，並推送到 [Google Container Registry](#)。使用上述的 IMAGE ID，看起來會像這樣

```
docker tag 95b4267bb6d0 gcr.io/${PROJECT_ID}/aspnetapp:alpine
docker push gcr.io/${PROJECT_ID}/aspnetapp:alpine
```


步驟 7 · 約 2 分鐘

7. 重新部署更新後的應用程式

編輯 YAML 檔案

切換回儲存 `dotnet-app.yaml` 檔案的目錄。在 YAML 檔案中找出以下這行程式碼

```
image: mcr.microsoft.com/dotnet/core/samples:aspnetapp
```

您需要變更此設定，以參照上方建立並推送至 gcr.io 的容器映像檔。

```
image: gcr.io/PROJECT_ID/aspnetapp:alpine
```

別忘了修改這個函式，以便使用您的 `PROJECT_ID`。完成後，畫面看起來大致如下：

```
image: gcr.io/myproject/aspnetapp:alpine
```

套用更新後的 YAML 檔案

```
$ kubectl apply -f dotnet-app.yaml
deployment.apps/dotnet-deployment configured
service/dotnet-service unchanged
```

請注意，「Deployment」資源顯示已更新，而「Service」資源則顯示未變更。您可以使用 `kubectl get pod` 指令查看更新後的 Pod，但這次我們會新增 `-w`，以便監控所有變更。

```
$ kubectl get pod -w
```

NAME	READY	STATUS	RESTARTS	AGE
dotnet-deployment-5c9d4cc4b9-cspqd	1/1	Running	0	34m
dotnet-deployment-5c9d4cc4b9-httw6	1/1	Running	0	34m
dotnet-deployment-5c9d4cc4b9-vvdlm	1/1	Running	0	34m
dotnet-deployment-85f6446977-tmbdq	0/1	ContainerCreating	0	4s
dotnet-deployment-85f6446977-tmbdq	1/1	Running	0	5s
dotnet-deployment-5c9d4cc4b9-vvdlm	1/1	Terminating	0	34m
dotnet-deployment-85f6446977-lcc58	0/1	Pending	0	0s
dotnet-deployment-85f6446977-lcc58	0/1	Pending	0	0s
dotnet-deployment-85f6446977-lcc58	0/1	ContainerCreating	0	0s
dotnet-deployment-5c9d4cc4b9-vvdlm	0/1	Terminating	0	34m
dotnet-deployment-85f6446977-lcc58	1/1	Running	0	6s
dotnet-deployment-5c9d4cc4b9-cspqd	1/1	Terminating	0	34m
dotnet-deployment-85f6446977-hw24v	0/1	Pending	0	0s
dotnet-deployment-85f6446977-hw24v	0/1	Pending	0	0s
dotnet-deployment-5c9d4cc4b9-cspqd	0/1	Terminating	0	34m
dotnet-deployment-5c9d4cc4b9-vvdlm	0/1	Terminating	0	34m
dotnet-deployment-5c9d4cc4b9-vvdlm	0/1	Terminating	0	34m
dotnet-deployment-85f6446977-hw24v	0/1	Pending	0	2s
dotnet-deployment-85f6446977-hw24v	0/1	ContainerCreating	0	2s
dotnet-deployment-5c9d4cc4b9-cspqd	0/1	Terminating	0	34m
dotnet-deployment-5c9d4cc4b9-cspqd	0/1	Terminating	0	34m
dotnet-deployment-85f6446977-hw24v	1/1	Running	0	3s
dotnet-deployment-5c9d4cc4b9-httw6	1/1	Terminating	0	34m
dotnet-deployment-5c9d4cc4b9-httw6	0/1	Terminating	0	34m

上述輸出內容顯示滾動式更新的過程。首先，系統會啟動新容器，並在容器執行時終止舊容器。

確認正在執行的應用程式

此時應用程式已更新完成，可以處理使用者要求。與先前一樣，您可以使用 Proxy 存取。

```
$ kubectl proxy --port 8080
Starting to serve on 127.0.0.1:8080
```

現在請使用 Cloud Shell 中的「網頁預覽」功能存取網頁應用程式。

網頁預覽操作說明請參閱步驟 4

在網頁預覽產生的網址中加入 `/api/v1/namespaces/default/services/dotnet-service:8080/proxy/`。最終看起來會像這樣：

```
https://8080-cs-473655782854-default.us-central1.cloudshell.dev/api/v1/namespaces/default/services/dotnet-service:8080/proxy/
```

確認 Kubernetes 服務是否正在分配負載

多次重新整理這個網址，您會發現主機有所變更，因為服務會將要求在不同 Pod 之間進行負載平衡。將主機值與上述 Pod 清單進行比較，確認所有 Pod 都收到流量。

擴充執行個體

在 Kubernetes 中調度應用程式資源非常簡單。下列指令會將 Deployment 擴充至 6 個應用程式執行個體。

```
$ kubectl scale deployment dotnet-deployment --replicas 6
deployment.apps/dotnet-deployment scaled
```

您可以使用這項指令查看新 Pod 及其目前狀態

```
kubectl get pod -w
```

請注意上方指令中的 `-w`。也就是「監看」指令，在本例中，這可讓您輕鬆查看 Pod 的建立和上線情形。

請注意，重新整理同一個瀏覽器視窗會顯示，流量現在已在所有新 Pod 之間平衡分配。

8. 恭喜！

在本實驗室中，我們在開發人員環境中驗證 .NET Core 範例網頁應用程式，然後使用 GKE 將其部署至 Kubernetes。接著修改應用程式，顯示執行所在容器的主機名稱。接著，Kubernetes 部署作業更新為新版本，並擴充應用程式，示範負載如何在額外執行個體之間分配。

如要進一步瞭解 .NET 和 Kubernetes，請參閱下列教學課程。這些內容會以本實驗室所學為基礎，介紹 Istio 服務網格，以便瞭解更精細的轉送和復原模式。

- <https://codelabs.developers.google.com/codelabs/cloud-istio-aspnetcore-part1#0>
 - <https://codelabs.developers.google.com/codelabs/cloud-istio-aspnetcore-part2#0>
-

步驟 9 · 約 2 分鐘

9. 清除所用資源

為避免產生非預期費用，請使用下列指令刪除本實驗室中建立的叢集和容器映像檔。

```
gcloud container clusters delete dotnet-cluster --zone ${DEFAULT_ZONE}
gcloud container images delete gcr.io/${PROJECT_ID}/aspnetapp:alpine
```
